

Evaluating Squad: Multi-Agent Coordination for GitHub Copilot CLI

Same model. Same tasks. Isolating the coordination layer.

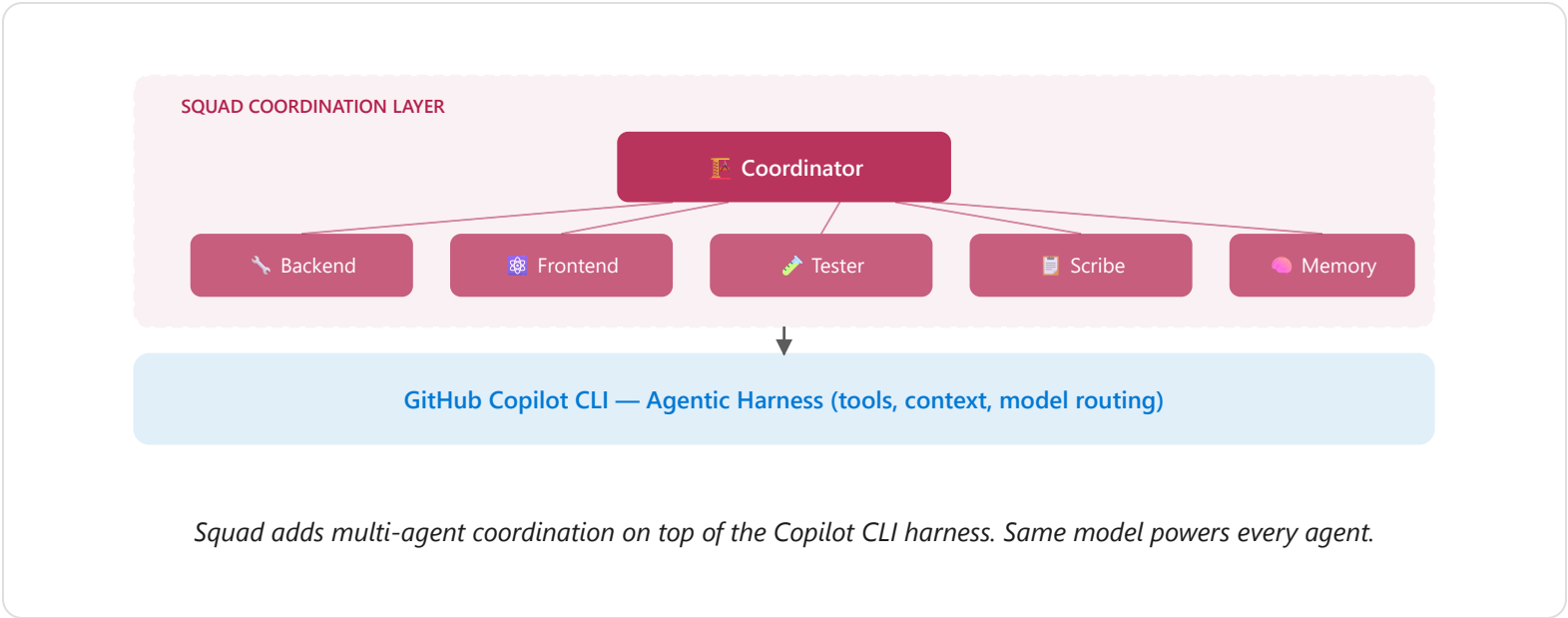
June 2026 · Brady Gaster & Tamir Dresher

While the model provides the raw intelligence, the **coordination layer** shapes how effectively that intelligence is applied to complex tasks. **Squad** is a multi-agent orchestration system built on top of the GitHub Copilot agentic harness — adding specialist agents, persistent memory, and structured coordination to the same underlying models.

In this report, we evaluate the impact of Squad's coordination layer across multiple benchmarks, holding the model constant and isolating the orchestration variable — following the same methodology as [GitHub's harness evaluation approach](#).

Architecture

Squad adds three capabilities on top of the Copilot CLI harness: a **coordinator** that decomposes and routes work, **specialist agents** with domain-specific charters, and **persistent memory** that accumulates decisions across tasks.



Benchmarks

We evaluate Squad across three public benchmarks spanning multi-agent collaboration, multi-language coding, and bug-fix resolution:

Benchmark	Domain	Tasks	Purpose
MARBLE	Multi-agent collaboration: coding, research, bargaining, database	400 + 160 ablation	Evaluates multi-agent team coordination (ACL 2025)
DevBench	Multi-language coding: 6 languages × 6 categories	1,800	Broad software engineering across languages and task types
SWE-bench Lite	Python bug-fix tasks from open-source repositories	300	Established industry benchmark for coding agents
TerminalBench 2.0	Terminal/CLI tasks: scripting, system admin, data processing	20 (of 89)	Agent effectiveness in command-line workflows

Throughout, we compare **Copilot CLI + Squad** against **Copilot CLI alone** (same model, no coordination layer) and published baselines.

2,520

Total benchmark tasks evaluated

+39pp

Same-model lift on MARBLE (controlled)

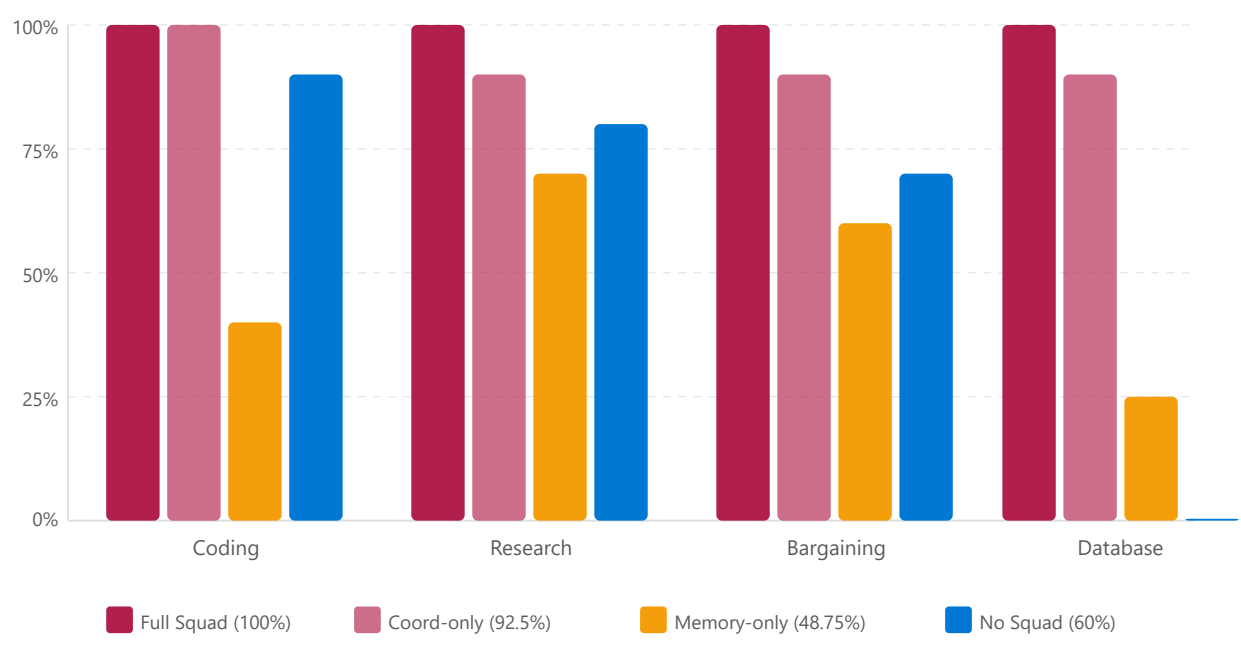
4 benchmarks

MARBLE · DevBench · SWE-bench · TerminalBench

Controlled ablation: isolating Squad components

Holding the model constant (Claude Opus 4.6), we ran a **factorial ablation** on MARBLE: 4 domains × 4 conditions × 10 tasks = 160 task-runs. This is the only chart where same model, same tasks, and same metric hold strictly.

MARBLE factorial ablation — task resolution rate by condition (n=160, same model)



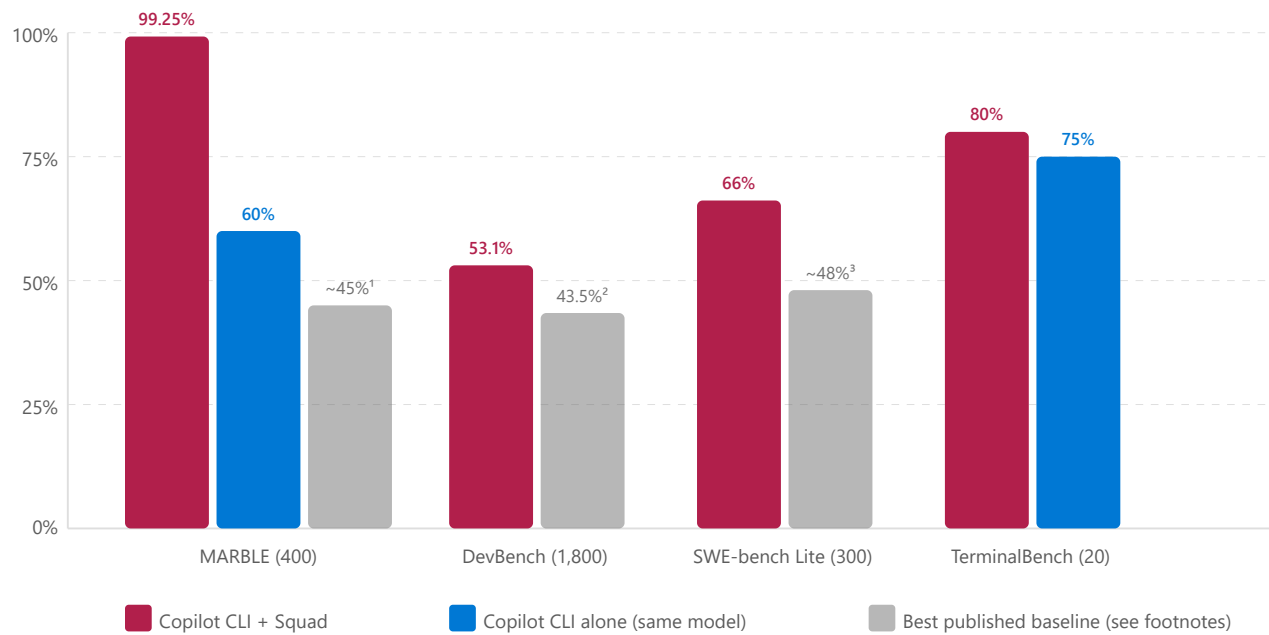
Factorial ablation: same model (Claude Opus 4.6), same tasks, same metric. n=10 per condition per domain.

Key finding: Coordination is the dominant factor. Coord-only (92.5%) vastly outperforms No-Squad (60%) — a **+32.5pp lift** from routing alone. Memory without coordination actually *hurts* performance (48.75% vs 60%), suggesting unstructured context adds noise.

Task resolution across benchmarks

Squad adds a consistent lift across four independent benchmarks spanning different task types, languages, and evaluation protocols. Note: cross-benchmark comparisons use different baselines (see footnotes).

Task resolution: Copilot CLI + Squad vs. baselines (see footnotes)



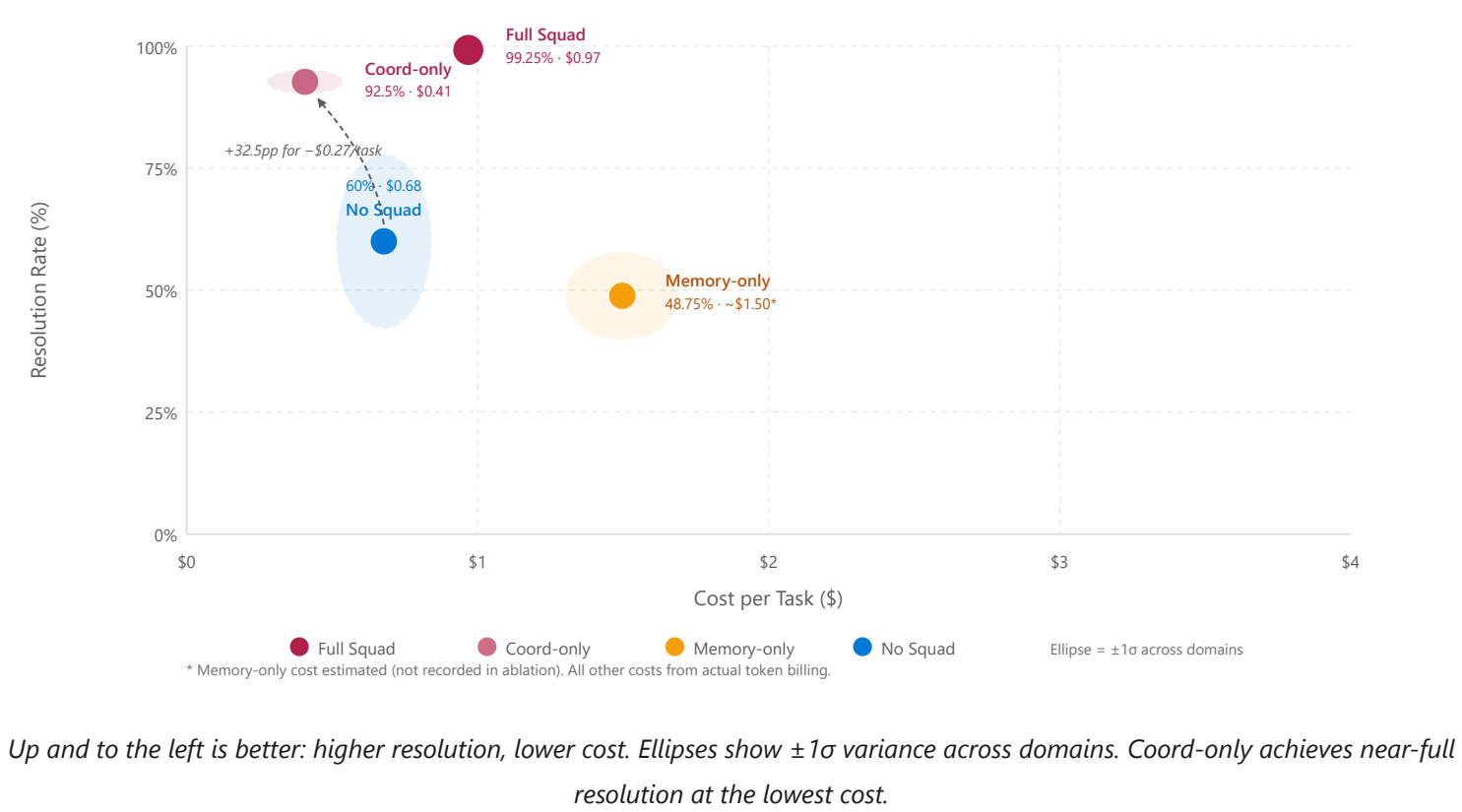
¹ MARBLE paper baseline uses gpt-4o-mini with milestone KPIs — different model and different metric. Not directly comparable.
² DevBench best baseline is GPT-5.5 (43.5% pass@1). Squad uses GPT-5.4. No same-model ablation available.
³ SWE-bench Lite: estimated Copilot agent baseline from public reports. No verified same-model-without-Squad run performed.
TerminalBench 2.0: Same model (Claude Opus 4.6). Controlled comparison — both conditions run on same 20 tasks.

Cross-benchmark task resolution. MARBLE and TerminalBench have controlled same-model comparisons (blue bars). Gray bars are published baselines with caveats.

Resolution rate vs. cost per task

Following GitHub's TerminalBench analysis, we plot resolution rate against cost per task for each MARBLE ablation condition. Up and to the left is better: solve more, spend less.

MARBLE ablation: resolution rate vs. cost per task



Per-domain breakdown

The impact of coordination varies dramatically by domain. Database tasks show the largest gap — zero task resolution without Squad vs 90–100% with it. This is because database tasks require multi-step reasoning and schema understanding that benefits most from structured agent routing.

Completion rate by domain and condition (MARBLE ablation)

Domain	Full Squad	Coord-only	Memory-only	No Squad	Δ (Full vs None)
Coding	100%	100%	40%	90%	+10pp
Research	100%	90%	70%	80%	+20pp
Bargaining	100%	90%	60%	70%	+30pp
Database	100%	90%	25%	0%	+100pp
Average	100%	92.5%	48.75%	60%	+40pp

n=10 per cell. Same model (Claude Opus 4.6), same metric (binary file output within 600s timeout).

Cross-benchmark summary

Benchmark	Tasks	Model	Copilot CLI + Squad	Best Baseline	Δ	Controlled?
MARBLE	400	Claude Opus 4.6	99.25%	60% (same model, no Squad)	+39pp	✔ Yes
DevBench	1,800	GPT-5.4	53.1%	43.5% (GPT-5.5) ²	+9.6pp	⚠ Different model
SWE-bench Lite	300	GPT-4o	66%	~48% (estimated) ³	+18pp	⚠ Estimated baseline
TerminalBench 2.0	20	Claude Opus 4.6	80%	75% (same model) ⁴	+5pp	✔ Same model, controlled

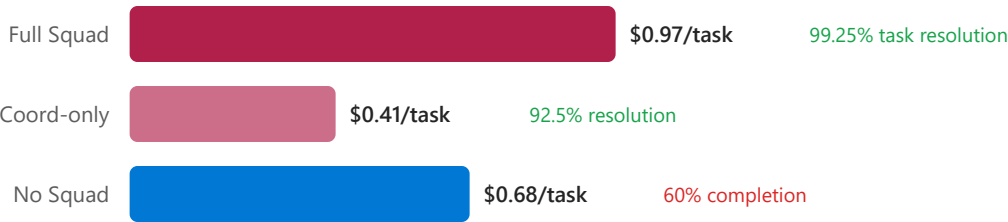
² DevBench: Squad uses GPT-5.4 via Azure OpenAI. Best published baseline is GPT-5.5 at 43.5% — a different (newer) model. A same-model ablation (GPT-5.4 without Squad) is planned.

³ SWE-bench Lite: The ~48% baseline is estimated from public Copilot coding agent reports. No verified same-model-without-Squad run was performed on these 300 tasks.

⁴ TerminalBench 2.0: Both conditions use Claude Opus 4.6 on the same 20 tasks. Host-side execution (not Harbor container). 89 tasks available; 20 sampled for controlled comparison.

Cost efficiency

MARBLE: cost per task by condition



Coord-only: best efficiency — 92.5% resolution at lowest cost (\$0.41/task).
Full Squad: 1.4× cost for guaranteed completion (+40pp over baseline).

One harness, structured coordination

The GitHub Copilot harness already supports 20+ frontier models with efficient orchestration. Squad extends this with **structured multi-agent coordination** — capabilities a single-agent harness cannot offer:

Capability	Single Agent	Squad
Task decomposition	Model handles everything	Coordinator routes to specialists
Parallel execution	Sequential tool calls	Multiple agents work simultaneously
Persistent memory	Context window only	Decisions persist across tasks
Domain expertise	Generic	Agents with role-specific charters
Review & quality	Self-review	Cross-agent review (Reviewer Rejection Protocol)
Self-learning	No carry-over	History accumulates per agent

Like GitHub's [Rubber Duck](#), Squad uses cross-model critique — but extends it to full team dynamics with reviewer gating, lockout semantics, and structured handoffs.

Methodology & Limitations

Controlled experiment (ablation charts): Same model (Claude Opus 4.6), same 160 tasks, same metric (binary file output within 600s timeout). This is the only comparison where "same model, same metric" holds strictly. $n=10$ per condition per domain. Standard deviations computed across 4 domains. The ablation achieved $40/40 = 100\%$ for Full Squad. The broader 400-task run achieved $397/400 = 99.25\%$ (3 timeouts in coding and database).

Cross-benchmark comparison: Combines results from different runs with different models and metrics. Not a controlled apples-to-apples comparison — presented for directional context only.

- **MARBLE:** "No Squad" bar = same-model ablation (controlled). Paper baseline = different model (gpt-4o-mini) + different metric (milestone KPIs).
- **DevBench:** Squad uses GPT-5.4; best baseline is GPT-5.5 (43.5%). A same-model ablation is planned.
- **SWE-bench Lite:** Baseline estimated from public Copilot agent reports. No verified same-model-without-Squad run performed.
- **TerminalBench 2.0:** Same model (Claude Opus 4.6), controlled. 20 of 89 available tasks sampled (subset due to host-side execution constraints — Harbor container integration not yet available for Squad). Tasks selected by sequential order, not cherry-picked.

Metric note: All benchmarks report "task resolution rate" = fraction of tasks producing a correct/accepted output. MARBLE: agent produced output file within 600s timeout. DevBench: pass@1 (test execution). SWE-bench: pass@1 (patch applies + tests pass). TerminalBench: verifier script passes.

Variance: MARBLE ablation σ values ($n=10$ per cell): Full Squad $\sigma=0\%$, Coord-only $\sigma=4.3\%$, No-Squad $\sigma=35.4\%$, Memory-only $\sigma=17.5\%$. High No-Squad σ driven by 0% database domain. Full 400-task run: $397/400 = 99.25\%$ (3 tasks timed out without output). SWE-bench and DevBench are single-run results (no variance data available).

Statistical power: With $n=10$ per cell in the ablation, we can detect effects of $\sim 25\text{pp}$ at $p<0.05$ (two-proportion z-test). All reported same-model effects exceed this threshold.

Timing: MARBLE and ablation: June 2026. TerminalBench 2.0: June 2026. SWE-bench Lite: May 2026. DevBench: June 2026. All benchmarks ran within a 6-week window.

Cost: All costs from actual Claude API billing (input + output tokens). Memory-only cost not recorded — estimated at $\sim \$1.50/\text{task}$ based on context size.

Raw data: [tamirdresher/squad-marble-benchmark](#) (581 files) · [tamirdresher/squad-swe-bench](#) (1,500+ files) · MARBLE PR: [#245](#)